Fundamental Algorithm Techniques - Problem Set #7

Problem 1: Graph Play

(1), (2), and (4) Graph Examples

Please see the Python code cell below for the generated diagrams of the transposed graph, the inverse (complement) graph, and the dual graph.

(3) What happens to the inverse graph if the original graph is dense?

If the original graph is very dense, meaning that the number of edges is close to the maximum possible, then its inverse (or complement) graph will be very sparse. The reason is simple: the complement graph contains only those edges that are missing in the original graph. So the denser the original graph is, the fewer edges remain in its complement.

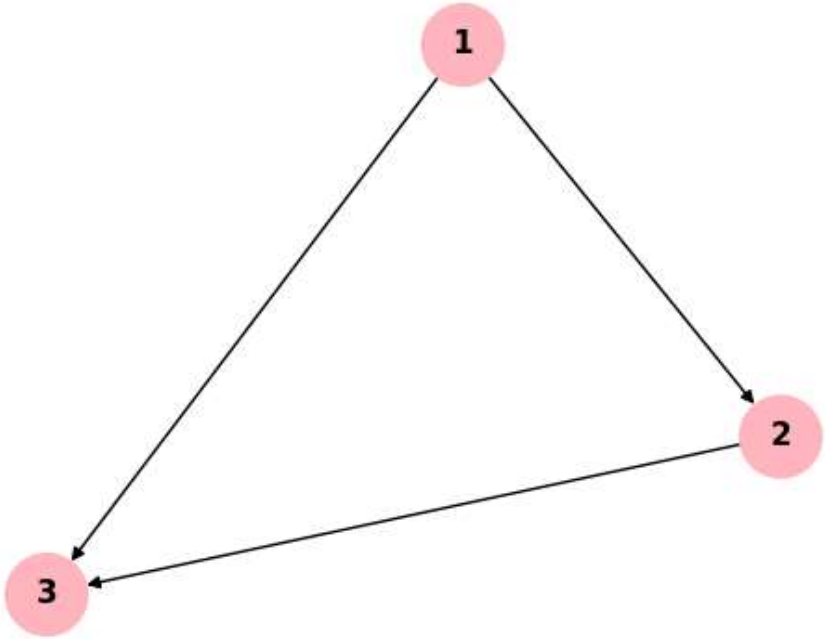
(5) Why is the dual graph only well-defined for planar graphs? Give an example of a non-planar graph.

The idea of a dual graph is based on turning each **face** of the original graph into a **vertex** in the dual graph. This only works when the graph is planar, because only a planar embedding divides the plane into well-defined faces. For a non-planar graph, such as (K_5) or (K_{3,3}), any drawing in the plane will necessarily have edge crossings. Once edges cross, the faces are no longer defined in a clean and unambiguous way, so the dual graph is not well-defined in that case.

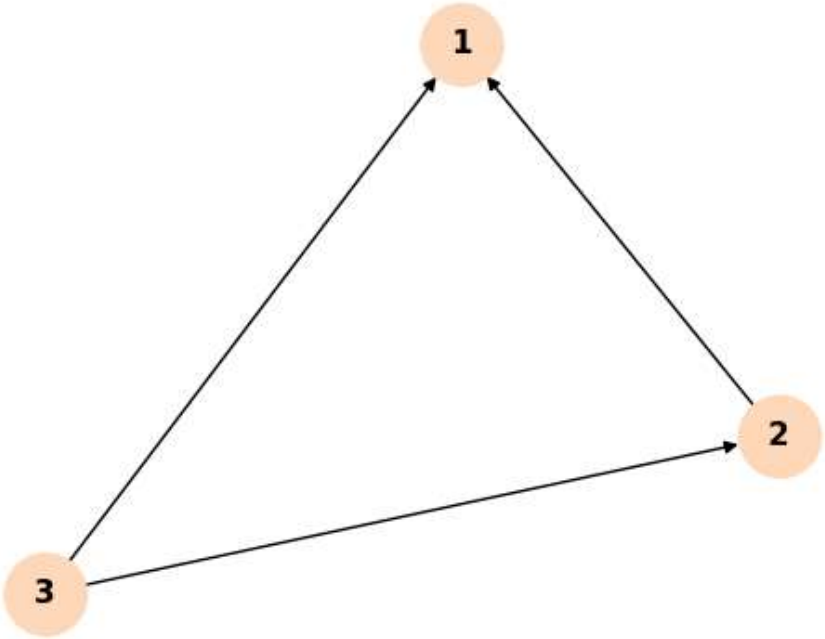
In [1]:
```
import networkx as nx
import matplotlib.pyplot as plt

fig, axes = plt.subplots(3, 2, figsize=(10, 12))

# 1. Directed Graph & Transposed Graph
G_dir = nx.DiGraph([(1, 2), (2, 3), (1, 3)])
G_trans = G_dir.reverse() # Transposed
```

```
pos_dir = nx.spring_layout(G_dir, seed=42)
nx.draw(G_dir, pos_dir, ax=axes[0,0], with_labels=True, node_color='#FFB6C1', node_size=1000, font_weight='bold', arrows=True)
axes[0,0].set_title("1a. Original Directed Graph")
nx.draw(G_trans, pos_dir, ax=axes[0,1], with_labels=True, node_color='#FFDAB9', node_size=1000, font_weight='bold', arrows=Tru
axes[0,1].set_title("1b. Transposed Directed Graph")

# 2. Undirected Graph & Inverse (Complement) Graph
G_undir = nx.Graph([(1, 2), (2, 3), (3, 4)])
G_inv = nx.complement(G_undir)
pos_undir = nx.circular_layout(G_undir)
nx.draw(G_undir, pos_undir, ax=axes[1,0], with_labels=True, node_color='#ADD8E6', node_size=1000, font_weight='bold')
axes[1,0].set_title("2a. Original Undirected Graph")
nx.draw(G_inv, pos_undir, ax=axes[1,1], with_labels=True, node_color='#90EE90', node_size=1000, font_weight='bold')
axes[1,1].set_title("2b. Inverse (Complement) Graph")

# 4. Simple Planar Graph & Dual Graph Concept (Triangle -> Star)
# A triangle has 1 inner face and 1 outer face. Its dual has 2 nodes connected by 3 edges.
G_planar = nx.Graph([('A', 'B'), ('B', 'C'), ('C', 'A')])
G_dual = nx.MultiGraph([('InnerFace', 'OuterFace'), ('InnerFace', 'OuterFace'), ('InnerFace', 'OuterFace')])
pos_planar = nx.spring_layout(G_planar, seed=42)
nx.draw(G_planar, pos_planar, ax=axes[2,0], with_labels=True, node_color='#E6E6FA', node_size=1000, font_weight='bold')
axes[2,0].set_title("4a. Simple Planar Graph (Triangle)")
nx.draw(G_dual, ax=axes[2,1], with_labels=True, node_color='#F0E68C', node_size=1000, font_weight='bold')
axes[2,1].set_title("4b. Its Dual Graph (Multigraph)")

plt.tight_layout()
plt.show()
```

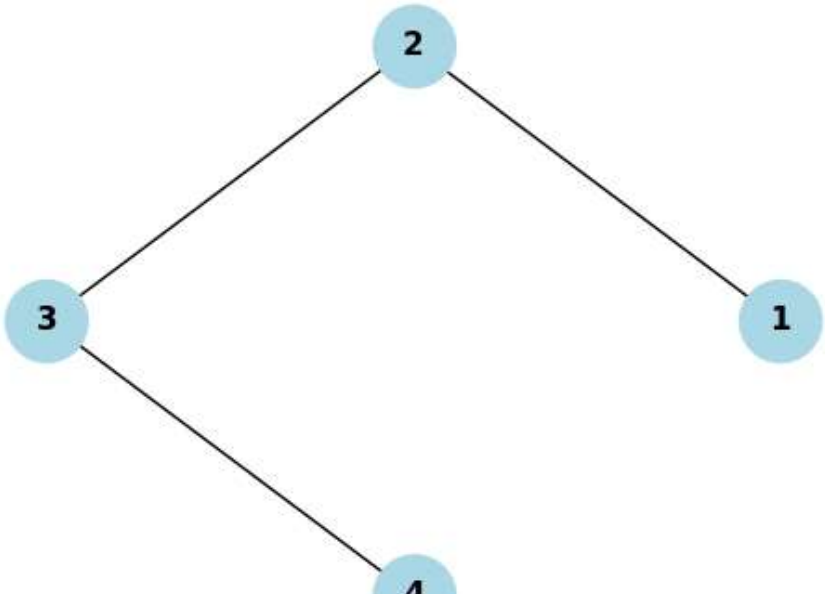
1a. Original Directed Graph



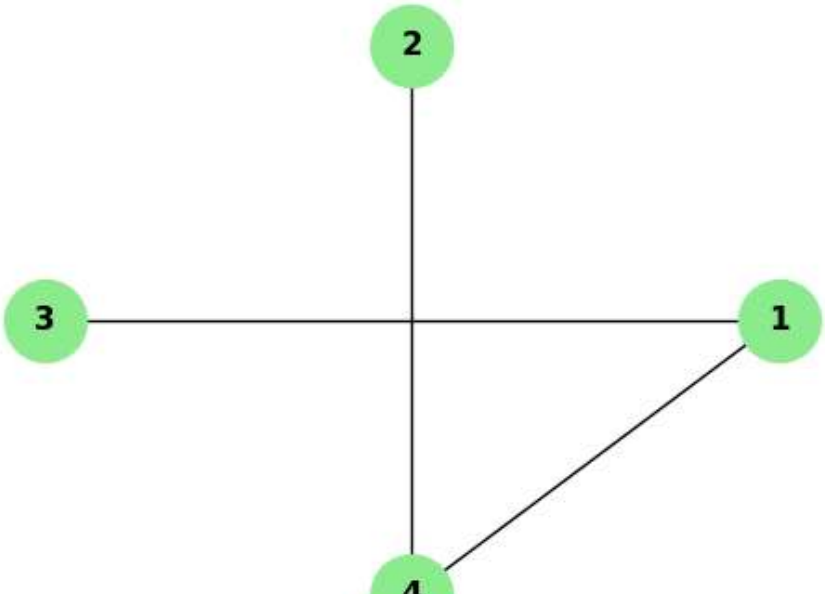
1b. Transposed Directed Graph

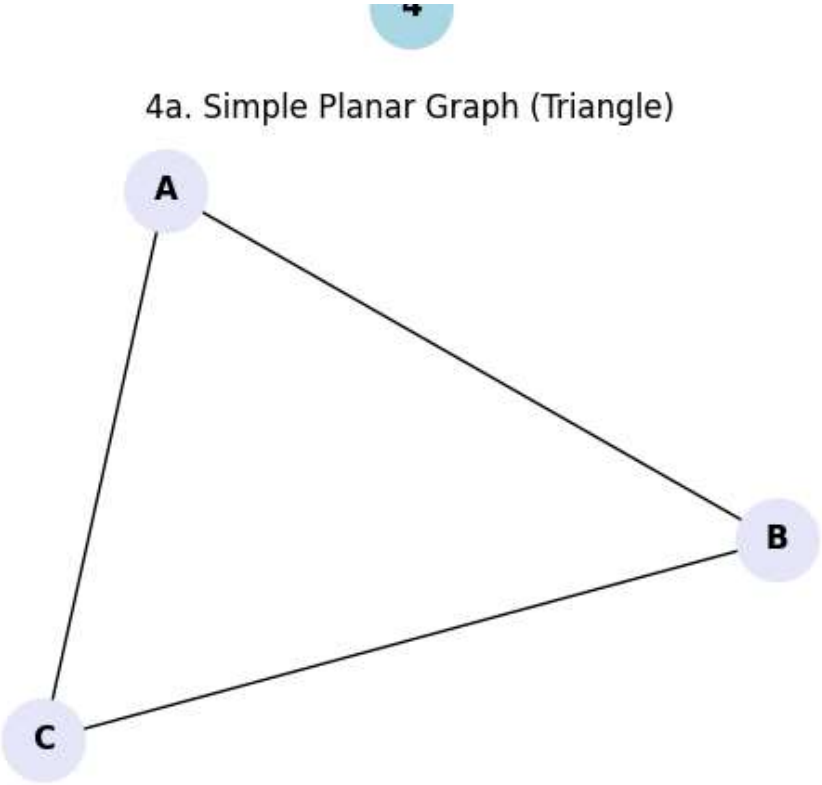


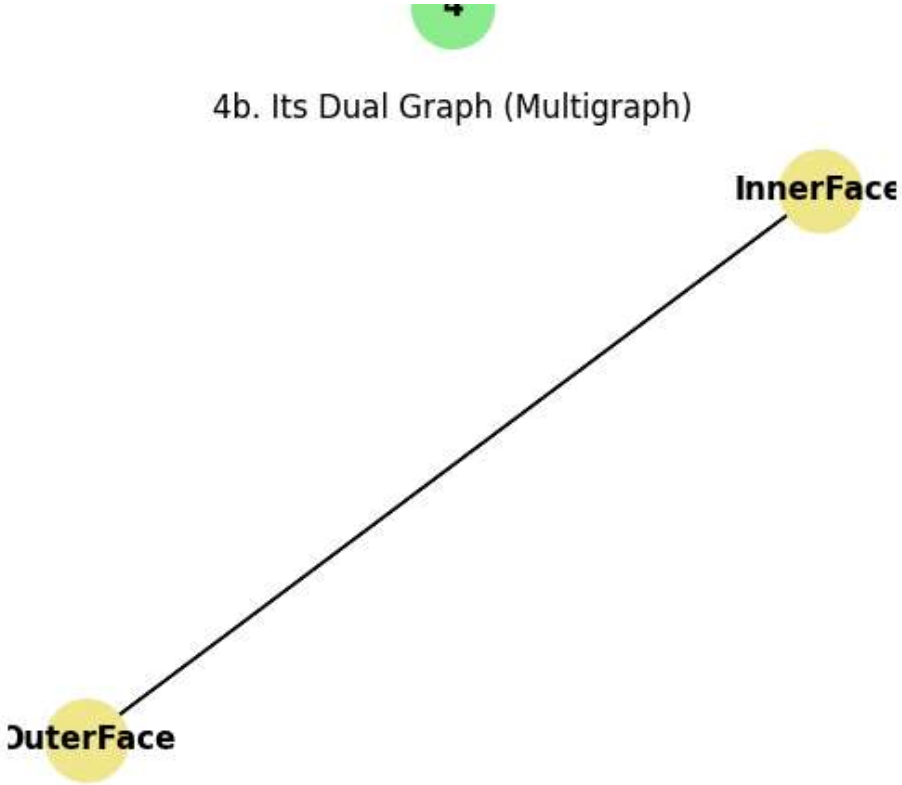
2a. Original Undirected Graph



2b. Inverse (Complement) Graph







Problem 2: Bron-Kerbosch Execution

Given graph G with $V = \{A, B, C, D\}$ and $E = \{AB, AC, BC, CD\}$. The neighborhood for each vertex is: $N(A) = \{B, C\}$, $N(B) = \{A, C\}$, $N(C) = \{A, B, D\}$, $N(D) = \{C\}$.

1. Initial Call:

- $R = \emptyset$
- $P = \{A, B, C, D\}$
- $X = \emptyset$

2. Trace the first two recursive calls (leading to a maximal clique): We start the loop in the initial call and pick vertex $v = A$.

- **Recursive Call 1 (Exploring from A):**
  - $R_1 = R \cup \{A\} = \{A\}$
  - $P_1 = P \cap N(A) = \{A, B, C, D\} \cap \{B, C\} = \{B, C\}$
  - $X_1 = X \cap N(A) = \emptyset \cap \{B, C\} = \emptyset$
  - **Values:** `(R={A}, P={B,C}, X=∅)`

Inside Call 1, we pick $v = B$ from P_1.

- **Recursive Call 2 (Exploring from B inside A):**
  - $R_2 = R_1 \cup \{B\} = \{A, B\}$
  - $P_2 = P_1 \cap N(B) = \{B, C\} \cap \{A, C\} = \{C\}$
  - $X_2 = X_1 \cap N(B) = \emptyset \cap \{A, C\} = \emptyset$
  - **Values:** `(R={A,B}, P={C}, X=∅)`

(Note: The subsequent Call 3 would pick C, resulting in $R = \{A, B, C\}$ and $P = \emptyset, X = \emptyset$, which reports the maximal clique `{A,B,C}`).

3. List of all maximal cliques & the maximum clique:

- All maximal cliques in graph G are: `{A, B, C}` and `{C, D}`.
- The **maximum clique** is `{A, B, C}` because it has the largest size (3 vertices compared to 2).

In [2]:
```
# Problem 2: Algorithmic Verification of Bron-Kerbosch

graph = {
    "A": {"B", "C"},
    "B": {"A", "C"},
    "C": {"A", "B", "D"},
    "D": {"C"}
}

maximal_cliques = []

def bron_kerbosch(R, P, X, step_label="Init"):
    # Convert sets to sorted lists just for cleaner printing
    print(f"[{step_label}] R={sorted(list(R))}, P={sorted(list(P))}, X={sorted(list(X))}")
```

```python
    if not P and not X:
        maximal_cliques.append(R)
        print(f"    --> Maximal Clique Found: {sorted(list(R))} \n")
        return

    for v in set(P): # iterate over a copy
        neighbors = graph[v]
        bron_kerbosch(R.union({v}), P.intersection(neighbors), X.intersection(neighbors), step_label=f"Explore {v}")
        P.remove(v)
        X.add(v)

print("--- Running Bron-Kerbosch Algorithm (Without Pivoting) ---")
bron_kerbosch(set(), {"A", "B", "C", "D"}, set(), step_label="Call 0")

print("--- Final Results ---")
print(f"All Maximal Cliques: {[sorted(list(c)) for c in maximal_cliques]}")

# Identify maximum clique
max_clique = max(maximal_cliques, key=len)
print(f"Maximum Clique(s): {sorted(list(max_clique))} (Size: {len(max_clique)})")
```

```
--- Running Bron-Kerbosch Algorithm (Without Pivoting) ---
[Call 0] R=[], P=['A', 'B', 'C', 'D'], X=[]
[Explore A] R=['A'], P=['B', 'C'], X=[]
[Explore B] R=['A', 'B'], P=['C'], X=[]
[Explore C] R=['A', 'B', 'C'], P=[], X=[]
    --> Maximal Clique Found: ['A', 'B', 'C']

[Explore C] R=['A', 'C'], P=[], X=['B']
[Explore C] R=['C'], P=['B', 'D'], X=['A']
[Explore B] R=['B', 'C'], P=[], X=['A']
[Explore D] R=['C', 'D'], P=[], X=[]
    --> Maximal Clique Found: ['C', 'D']

[Explore B] R=['B'], P=[], X=['A', 'C']
[Explore D] R=['D'], P=[], X=['C']
--- Final Results ---
All Maximal Cliques: [['A', 'B', 'C'], ['C', 'D']]
Maximum Clique(s): ['A', 'B', 'C'] (Size: 3)
```

In [ ]: